W9 - *PRACTICE*

Going deeper with states

At the end of his practice, you should be able to…

- ✓ Reflect on **your usage of states**
- ✓ Style widgets dynamically
- ✓ **Pass functions** as value to props
- ✓ **Handles objects** in states
  - Treat state as **read-only**
  - Copy objects with the **spread syntax**

How to work?

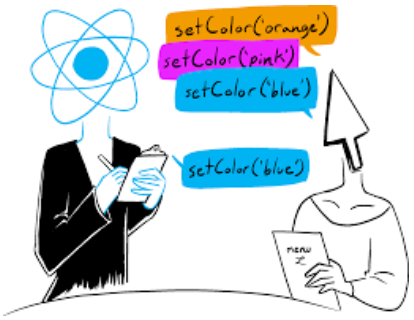
- ✓ Download **the start code** from the Google classroom
- ✓ For each exercise you can either:
  - Run `npm install`
  - Or move an existing `node_modules` to the exercise folder *(fastest option!)*

How to submit?

- ✓ **Create a repository on GitHub** with the name of this practice:
  Ex: C2-S6 PRACTICE
- ✓ **Push your final code** on this GitHub repository (if you are lost, follow this tutorial)
- ✓ Finally submit on **Google classroom** your GitHub repository URL
  Ex: `https://github.com/thebest/` C2-S6 PRACTICE`.git`

Are you lost?

You can read the following documentation to be ready for this practice:
https://www.w3schools.com/react/react_events.asp
https://react.dev/learn/updating-objects-in-state



Project Setup & Folder Structure

Before starting, ensure your `src` folder is organized like this:

- `src/components/` (Reusable UI components)
- `src/pages/` (Page-level components for Routing)
- `src/App.jsx` (Main entry point with Router)

Ex1: The Toggle Switch (Folder Structure & CSS Classes)

Goal: Practice toggling a state to dynamically apply CSS classes.

- **Location:** `src/components/ToggleBox/ToggleBox.jsx`
- **Task:** Complete the `handleToggle` function so that clicking the button adds/removes the `active` class.

```
// STARTUP CODE
import { useState } from 'react';
import './ToggleBox.css'; // Assume .active { background: blue; color: white; }

export default function ToggleBox() {
  const [isSelected, setIsSelected] = useState(false);

  const handleToggle = () => {
    // GAP: Update state to the opposite of its current value
  };

  return (
    <div className="p-4">
      <p className={isSelected ? "active p-2" : "p-2"}>Style me!</p>
      <button onClick={handleToggle} className="border p-1">
        {isSelected ? "Deactivate" : "Activate"}
      </button>
    </div>
  );
}
```

Ex2: The Conditional Delete Dialog (Logical &&)

Goal: Use conditional rendering to show/hide a modal and handle dismissals via callbacks.

- **Location:** `src/components/Modal/DeleteModal.jsx`
- **Task:** Show the warning box only when `isVisible` is true.

```
// STARTUP CODE
export default function DeleteModal() {
  const [isVisible, setIsVisible] = useState(false);

  return (
    <div className="p-5 border">
      <button onClick={() => setIsVisible(true)} className="bg-red-500 text-white p-2">
        DELETE ITEM
      </button>

      {/* GAP: Use the && operator to conditionally show the div below */}
      (
        <div className="mt-4 p-4 bg-yellow-100 border border-yellow-400">
          <p>Are you sure?</p>
          <button onClick={() => setIsVisible(false)} className="bg-gray-300 p-1">
            PROCEED (Dismiss)
          </button>
        </div>
      )
    </div>
  );
}
```

Ex3: The Async Counter (Functional State Updates)

Goal: Fix the "late by 1" state issue using an updater function.

- **Concept:** When updating state based on the *previous* state, always use the callback version of the setter.
- **Task:** Fix the `increment` function so it correctly adds 1 using the `prevCount` parameter.

```
// STARTUP CODE
export default function AsyncCounter() {
  const [count, setCount] = useState(0);

  const handleIncrement = () => {
    // GAP: Use the functional update pattern (prevCount => ...)
    // This ensures state is updated correctly even if asynchronous
    setCount(________________);
  };

  return (
    <div className="p-4">
      <h1>Count: {count}</h1>
      <button onClick={handleIncrement}>+1 (Reliable)</button>
    </div>
  );
}
```

Ex4: Selecting the "Best Joke" (Lifting State Up)

Goal: Pass a function as a prop so the child can update the parent's state.

- **Location:** `src/components/JokeList/JokeItem.jsx`
- **Task:** When a `JokeItem` is clicked, it should tell the parent `App` which ID was selected.

```
// CHILD COMPONENT (JokeItem.jsx)
const JokeItem = ({ joke, onSelect, isSelected }) => (
  <div
    onClick={() => onSelect(joke.id)}
    className={`p-4 border cursor-pointer ${isSelected ? "bg-blue-100 border-blue-500" : ""}`}
  >
    {joke.text}
  </div>
);

// PARENT COMPONENT (App.jsx)
const JOKES = [{ id: 1, text: "Joke A" }, { id: 2, text: "Joke B" }];

export default function App() {
  const [selectedId, setSelectedId] = useState(null);

  return (
    <div>
      {JOKES.map(j => (
        <JokeItem
          key={j.id}
          joke={j}
          isSelected={selectedId === j.id}
          // GAP: Pass the setter function as a callback
          onSelect={________________}
        />
      ))}
    </div>
  );
}
```

Ex5: Navigating the Lab (React Router)

Goal: Set up basic routing to switch between your exercise components.

- **Task:** Fill in the `path` and `element` props to link the components you just built.

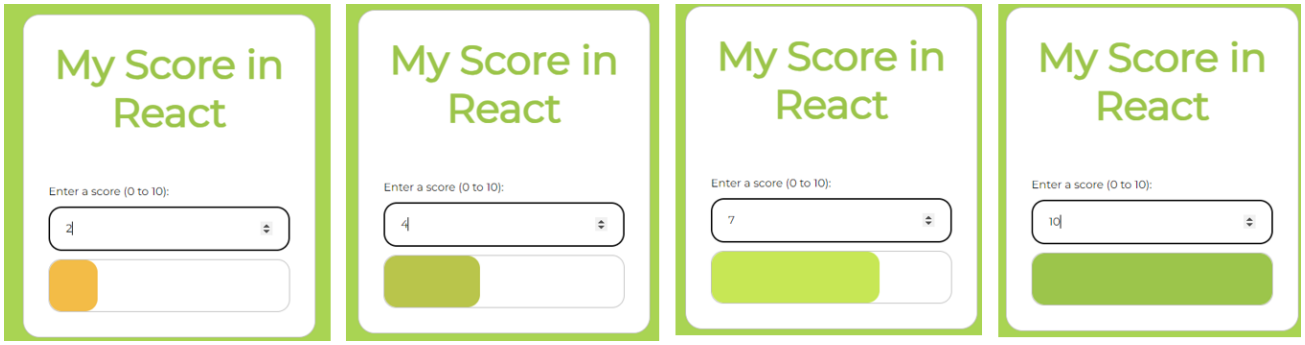
```
// STARTUP CODE
import { BrowserRouter, Routes, Route, Link } from 'react-router-dom';
// Import your components from /components/...

function App() {
  return (
    <BrowserRouter>
      <nav className="p-4 flex gap-4 bg-gray-800 text-white">
        <Link to="/">Toggle</Link>
        <Link to="/counter">Counter</Link>
      </nav>

      <Routes>
        {/* GAP: Define the Route for the Home path "/" */}
        <Route path="____" element={<ToggleBox />} />

        {/* GAP: Define the Route for the "/counter" path */}
        <Route path="____" element={<AsyncCounter />} />
      </Routes>
    </BrowserRouter>
  );
}
```

Ex6: (Startup code - EXERCISE 1)

You can click on this link to see the expected results for this exercise:
https://react-s11-ex1.vercel.app/

The goal of this app is to update the progress bar, according to the value entered in the input field:

- Values range are from 0 to 10
- Progress bar color changes depending on the score (see bellow)



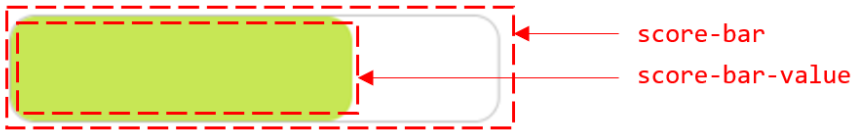
Depending on the score value (0 to 10) the progress bar with and color shall be updated

Step 1: What is/are the state(s) you plan to use for this exercise? Complete the table.

State Name	Type	Role	Used for…	Changed when…

Step 2: Update the **progress bar width** according to the score value

Tips: The progress has 2 div: the border rectangle and the filled rectangle. Which one should you update?



Step 3: Update the **progress bar color** according to the score value

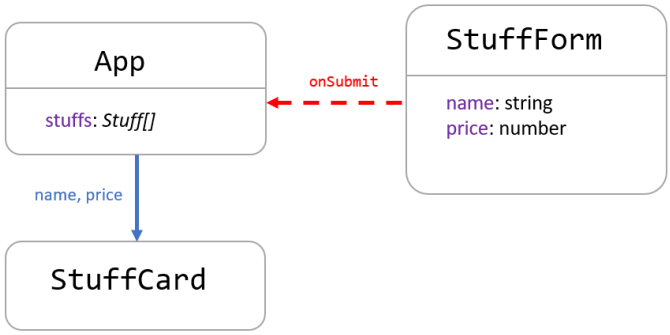
Ex7: (Startup code - EXERCISE 2)

You can click on this link to see the expected results for this exercise:
https://react-s11-ex2.vercel.app/

The user can specify an object name and price. By clicking on Add, this object shall be added to the list.

Step 1:
Look at the start code. We have 3 components.

- ✓ App manages a state with the list of objects
- ✓ StuffCard displays 1 object *(nothing to change on this component!)*
- ✓ StuffForm manages the form and send an event when user has click on Add



Complete the **StuffFrom** to handle the input values with the following states:

State	Type	Component	Used to…	Changed when…
name	String	StuffFrom	Store the entered name	Input name had changed
price	number	StuffFrom	Store the entered price	Input price had changed

Step 2:
When user clicks on Add button, the StuffFrom component sends an event to the App component
Complete the code to send this event, with the entered name and price as parameter

Note: For now, just log something on the console to check you can manage the event on App component. For example:

```
A new object named Piano, price 15$ will be added to the list
```

Step 3:
The last step is to update the state containing the stuff, with the new object.

State	Type	Component	Used to…	Changed when…
stuff	Array	App	Storing all stuffs	Add a new stuff

First read the following articles about **objects** and **arrays** in state:

- https://react.dev/learn/updating-objects-in-state
- https://react.dev/learn/updating-arrays-in-state

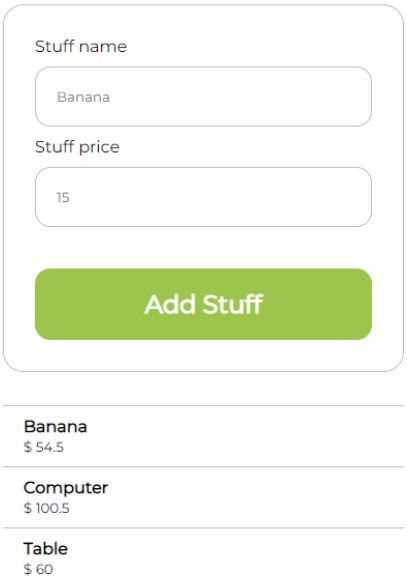
Q1 – Why states containing an array or an object **should be treated as read only**?

Q2 – Why do we need to **create a new array** when we want to add a created object to a state array?

Q3 – What is the **spread operator**? How can you use it to clone the state array and add the new created object?

Q4 – Then complete the code to update the state array with the new object.

The finished app will look like this:

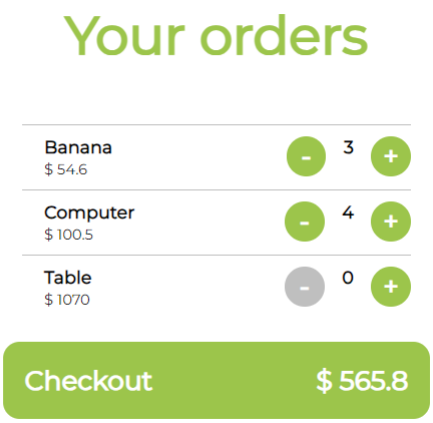


Ex8: (Startup code - EXERCISE 2)

You can click on this link to see the expected results for this exercise:
https://react-s11-ex3.vercel.app/

This app allows the user to adjust the quantity of items to buy and to compute the total price.
Note: the minus button is disabled when the quantity is equal to 0.

The finished app will look like this:



Step 1:
Look at the start code. We have 3 components.

- ✓ App manages a state with the list of items
- ✓ Order Card handle the quantity to order for 1 item
- ✓ CheckoutButton displays the total sum *(nothing to change on this component!)*

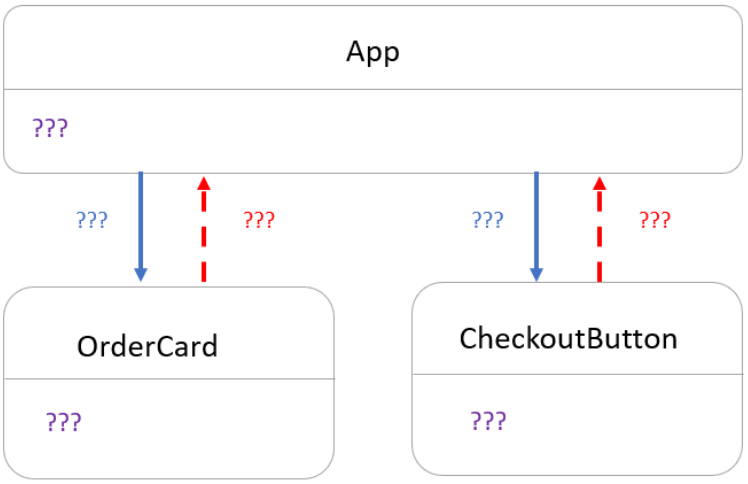
Q1 - What is/are the state(s) you plan to use for this exercise? Complete the table.

State Name	Type	Component	Used for…	Changed when…

Tips: Is the total price a state?

Q2 - What are the interactions you plan to define between the components?
Complete the diagram: *purple=states*, *blue=props data*, *red=prop event*

Tips: what's happen when you press + or – button on an item card? Which component will handle the state update?



Q3 – Update the code to display each item using `OrderCard` components (name, unit price and current quantity)

Note: For now, the + and – button do not work!

Q4 – Update the code to **send the right total price** to the `CheckoutButton`

Tips: It's always good to separate our code with clear functions!

Q5 – When user click on + or - the quantity of the related item must change, and also the total price

- Which state needs to be updated? Which component manages this state?

Tips: you don't know how to update a state array?
- Read again this document (section Replacing items in an array)
- Understand how to use the non-mutating methods `map` and spread operation to perform this operation.

Q6 – When quantity is equal to 0, the **minus button shall be disabled** on the order card.

To effectively disable the button, we need to display it gray but also prevent any action when clicking on it.

Therefore, when quantity is equal to 0:
- The minus button background is gray (`#bfbfbf`)
- No action can be done when clicking on minus button